

Informe: Implementación de motores de inferencia

En esta sección se describen, de manera narrativa y detallada, los pasos necesarios para implementar tres tipos de motores de inferencia en sistemas basados en conocimiento: **encadenamiento hacia adelante**, **encadenamiento hacia atrás** y **resolución (por contradicción)**. Se incluyen ejemplos simples de implementación para que quede más claro cómo se programan.

Definiciones clave

- **Hecho:** proposición atómica que se asume verdadera en la base de conocimiento (ej. `d`, `e`).
- **Regla:** implicación de la forma `premisas → conclusión` que permite derivar nuevos hechos.
- **Base de conocimiento (BK):** conjunto de hechos y reglas que representan el conocimiento disponible.
- **Cláusula:** en FNC, una disyunción de literales (ej. `¬b ∨ ¬c ∨ a`).
- **Literal:** un símbolo proposicional positivo (ej. `a`) o negado (ej. `¬a`).
- **Resolvente:** nueva cláusula obtenida al aplicar la regla de resolución a dos cláusulas que contienen literales complementarios (ej. de `a ∨ x` y `¬a ∨ y` obtenemos `x ∨ y`).

1. Motor de inferencia con encadenamiento hacia adelante

El encadenamiento hacia adelante es un proceso **progresivo** que parte de los hechos conocidos y busca derivar nuevas conclusiones hasta llegar a la hipótesis deseada.

Componentes necesarios:

1. **Base de conocimiento:** reglas de la forma `premisas → conclusión`. En programación se pueden guardar como listas o diccionarios. Ejemplo en Python:

```
reglas = [  
    {"premisas": ["b", "c"], "conclusion": "a"},  
    {"premisas": ["d", "e"], "conclusion": "b"}  
]
```

1. **Conjunto de hechos iniciales:** representados como un `set`.

```
hechos = {"d", "e"}
```

1. **Motor de inferencia:** aplica las reglas y genera nuevas conclusiones.

Algoritmo narrativo:

- Mientras se puedan derivar hechos nuevos, recorrer todas las reglas.
- Si las premisas de una regla están contenidas en los hechos, añadir la conclusión.

- Repetir hasta alcanzar el objetivo o no poder derivar más.

Ejemplo simplificado:

```
objetivo = "a"
nueva_inferencia = True

while nueva_inferencia:
    nueva_inferencia = False
    for regla in reglas:
        if all(p in hechos for p in regla["premisas"]):
            if regla["conclusion"] not in hechos:
                hechos.add(regla["conclusion"])
                nueva_inferencia = True
                if objetivo in hechos:
                    print("Objetivo probado")
```

2. Motor de inferencia con encadenamiento hacia atrás

El encadenamiento hacia atrás parte de la hipótesis y busca justificarla con reglas y hechos.

Componentes necesarios:

1. **Base de conocimiento:** como en el caso anterior.
2. **Hechos iniciales:** proposiciones que ya son verdaderas.
3. **Función recursiva:** que intenta probar el objetivo.

Ejemplo narrativo:

- Para probar un objetivo, primero revisar si ya está en los hechos.
- Si no, buscar reglas cuya conclusión sea ese objetivo.
- Para cada regla, intentar probar recursivamente todas sus premisas.
- Si todas se prueban, el objetivo se añade a los hechos.

Ejemplo simplificado:

```
hechos = {"d", "e"}

reglas = [
    {"premisas": ["b", "c"], "conclusion": "a"},
    {"premisas": ["d", "e"], "conclusion": "b"},
    {"premisas": ["e"], "conclusion": "c"}
]

def probar(objetivo):
```

```

    if objetivo in hechos:
        return True
    for regla in reglas:
        if regla["conclusion"] == objetivo:
            if all(probar(p) for p in regla["premisas"]):
                hechos.add(objetivo)
                return True
    return False

print(probar("a")) # Debería devolver True

```

3. Motor de inferencia con resolución (por contradicción)

Este método parte de la hipótesis negada y busca una contradicción aplicando resolución sobre cláusulas en FNC.

Componentes necesarios:

1. **Base de conocimiento en FNC:** cada regla como una lista de literales (ej. ["¬b", "¬c", "a"]).
2. **Hipótesis negada:** se agrega como nueva cláusula.
3. **Procedimiento de resolución:** combina cláusulas que contienen literales opuestos.

Ejemplo narrativo:

- Agregar la negación de la hipótesis.
- Tomar pares de cláusulas y aplicar resolución.
- Si aparece la cláusula vacía, hay contradicción y el objetivo se prueba.

Ejemplo simplificado:

```

clausulas = [
    ["¬b", "¬c", "a"],
    ["¬d", "¬e", "b"],
    ["d"],
    ["e"],
    ["¬e", "c"]
]

# Agregar la negación del objetivo
clausulas.append(["¬a"])

# Función de resolución básica
def resolver(c1, c2):
    resolventes = []
    for lit in c1:

```

```
opuesto = lit[1:] if lit.startswith("¬") else "¬"+lit
if opuesto in c2:
    nueva = list(set(c1 + c2))
    nueva.remove(lit)
    nueva.remove(opuesto)
    resolventes.append(nueva)
return resolventes
```

En un programa completo habría que iterar sobre todos los pares de cláusulas hasta encontrar la cláusula vacía.

Conclusiones

- El **encadenamiento hacia adelante** se implementa con un bucle iterativo que añade hechos.
- El **encadenamiento hacia atrás** se implementa naturalmente con recursión.
- La **resolución** requiere transformar reglas a FNC y aplicar la regla de resolución sistemáticamente.

Con estos ejemplos básicos se ve claramente cómo pasar de la teoría a una implementación en programación.

Comparación de los métodos

- **Encadenamiento hacia adelante:** conviene usarlo cuando se tienen muchos hechos iniciales y se quiere derivar todas las consecuencias posibles o verificar si se alcanza cierto objetivo en el camino. Es útil en sistemas expertos de diagnóstico. Ventaja: simple de implementar y eficiente cuando hay abundancia de datos. Converge rápido si el objetivo aparece temprano en las derivaciones.
- **Encadenamiento hacia atrás:** adecuado cuando se tiene una hipótesis concreta que probar y no se desea explorar todas las consecuencias de la base. Es típico en sistemas de consulta (ej. motores de reglas tipo Prolog). Ventaja: evita generar información irrelevante. Converge más rápido si las reglas que llevan al objetivo son pocas y claras.
- **Resolución (por contradicción):** se usa cuando se requiere un método general y formalmente completo para probar teoremas. Es la base de los demostradores automáticos de teoremas. Ventaja: garantiza que, si la conclusión se sigue lógicamente, el método lo puede encontrar. Puede ser más costoso computacionalmente que los otros métodos.

Resumen:

- **Forward chaining** → mejor cuando hay muchos hechos y queremos descubrir todo lo derivable.
- **Backward chaining** → mejor cuando hay un objetivo específico que comprobar.
- **Resolución** → más general y completo, aunque más pesado; asegura demostraciones formales.